

W kierunku najlepszej dokładności

Stanisław Frelik

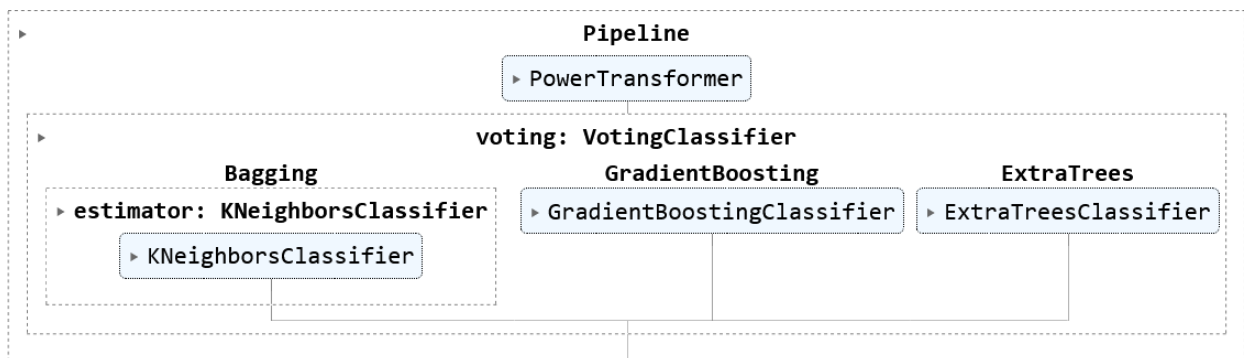
8 stycznia 2024

1 Cel pracy

Cel pracy jest bardzo prosty - jak najbardziej 'zbalansowana' dokładność. Jak go osiągnąć? Opowiemy o tym więcej w kolejnych rozdziałach. Zapraszamy!

2 Konstrukcja

Kontynuując ideę prostego celu pracy postanowiliśmy zastosować 'proste' podejście do postawionego problemu. Nie implikuje to jednak tego, iż 'proste' podejście wymagało mało pracy. Aby uzyskać jak największą zdolność predykcyjną zdecydowaliśmy się na budowę modelu klasyfikacji w duchu metod *state of the art*. Schemat wybranego modelu prezentuje poniższa grafika:



W celu uzyskania jak największej zdolności predykcyjnej wybraliśmy trzy klasyfikatory:

- *GradientBoostingClassifier*
- *ExtraTreesClassifier*
- *KNeighborsClassifier*

Następnie dostroiliśmy ich hiperparametry do naszego problemu, ostatnią metodę wzmocniliśmy poprzez *BaggingClassifier* i całość zamknęliśmy w *VotingClassifier*. Szczegóły przebiegu tej konstrukcji opisane zostały w rozdziale **Eksperyment**. Wszystkie wykorzystane przez nasz narzędzia można więc zaliczyć do metod opartych na komitetach. Taki wybór podyktowany był ich skutecznością. Więcej o napotkanych problemach i podjętych (trudnych) decyzjach napominamy w ostatnim rozdziale **Podsumowanie**.

3 O danych

Jako aspirowujący *data scientists* grzechem byłoby pominięcie wnikliwej analizy otrzymanych do pracy danych. Tym bardziej jest to istotnie, ponieważ zostały one sztucznie wygenerowane, nie otrzymaliśmy żadnych wskázówek co do prawdziwej tożsamości badanych cech. Stąd trudno o jakiegokolwiek intuicje - potrzebne będą nam za to metody do ich uzyskania działające 'po omacku'. Pierwsze spojrzenie na dane napawało optymizmem. Nie występowały żadne braki czy obserwacje odstające mogące wpłynąć negatywnie na zdolności predykcyjne

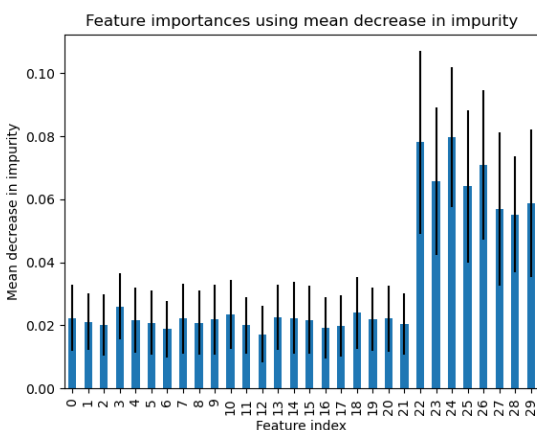
wytrenowanego modelu. Ponadto po szybkiej, wizualnej inspekcji okazało się, że większość cech jest stosunkowo 'normalnie' rozłożona, a każda z klas równomiernie reprezentowana - kolejny dobry omen. Następnym logicznym krokiem było zadbanie o przeskalowanie wszystkich cech. Jest to pożyteczna praktyka pozytywnie wpływająca na zachowanie modeli uczenia maszynowego (w szczególności *KNeighborsClassifier*). Pozostaje pytanie jakiego 'scalera' użyć. Zgodnie ze schematem z poprzedniego rozdziału zdecydowaliśmy się na użycie *PowerTransformer*. Przyczyn takiego wyboru było kilka:

- *PowerTransformer* 'stara się' aby przeskalowane dane były 'bardziej Gaussowskie' co w połączeniu z przekonaniem, iż nasze dane są już 'stosunkowo normalne' może dać zadowalające efekty
- zgłębienie dokumentacji *scikit-learn* oraz artykułów wskazywało na przewagę tego scalera nad innymi [1]
- sama nazwa **PowerTransformer** sugeruje, iż jest to potężne narzędzie, które na pewno pozytywnie wpłynie na zdolność predykcyjną naszego modelu

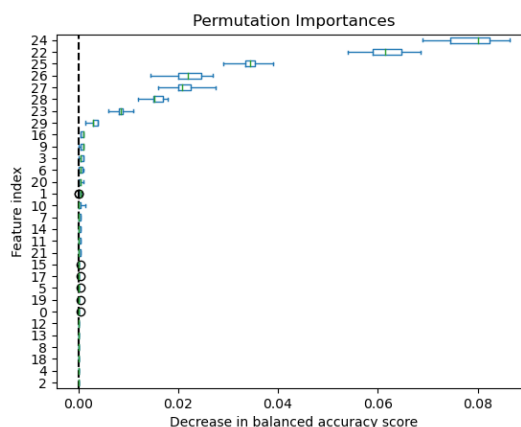
3.1 O istotności cech

'Na chłopski rozum zdawałoby się, że im więcej danych wrzucimy do modelu, tym lepsza powinna być jego zdolność predykcyjna.'

Jak się jednak (mamy nadzieję) okazało - nic bardziej mylnego. Ostatnim z rozważanych w tej pracy zabiegów na surowych danych jest ich selekcja, czyli wybór tych faktycznie istotnych (pod względem zwiększania zdolności predykcyjnej). Procedura selekcji jest istotnie nietrywialna, trudno doszukiwać się wręcz czegoś takiego jak właśnie 'procedura', a standardowe narzędzia inspekcji niekoniecznie są zdolne do wykazania istotności poszczególnych cech. Tym bardziej jest to skomplikowane, gdy nie posiadamy innej intuicji (odnosimy się do tego, że oczywistym jest na przykład iż port, w którym pasażer wsiadał na *Titanica* nie powinien wpłynąć na to, czy przeżył niesławną katastrofę). Niemniej mamy do dyspozycji pewne techniki, które pozwalają na ocenę istotności cech otrzymanych obserwacji. Pierwszym pomysłem na takową technikę była (zasłyszana na wykładzie :)) cecha modelu *RandomForest*. Jak się okazuje ów model posiada zdolność estymacji istotności cech (w szczególności dla modelu z pakietu *scikit-learn* *RandomForestClassifier* dostępna jest metoda *features_importances_*). W połączeniu z tym, że model lasu losowego jest stosunkowo 'dobrym' modelem uczenia maszynowego postanowiliśmy skorzystać z jego 'dobroci' i 'uciąć' troszkę cech. Wyniki tej estymacji istotności widoczne są na Rysunku 1. Na osi pionowej prezentujemy 'mean decrease in impurity' - czyli jak poszczególne cechy średnio wpływają na zwiększanie jakości modelu (niebieskie słupki) oraz ich niestabilność (czarne paski).



Rysunek 1: RandomForest



Rysunek 2: Permutation_Importance

Metoda MDI (Mean Decrease in Impurity) jest skuteczna, jednak jak zaznaczono między innymi w [2] cierpi na pewne mankamenty, na przykład 'przecenia' cechy o licznych nośnikach. Ponadto w [3] znajdujemy

propozycję rozwiązania naszego potencjalnego problemu - skorzystanie z funkcji *Permutation_importance*. Umożliwia ona selekcję zmiennych poprzez permutowanie cech i ocenę ich wpływu na jakość modelu - proces zbliżony do krosvalidacji. Oczywiście funkcja *Permutation_importance* przyjmuje estymator (czyli pewien model) potrzebny do generowania predykcji na poszczególnych permutacjach. Logicznym wydaje się użycie takiego modelu, który sam w sobie prowadzi do selekcji zmiennych - dzięki takiemu wyborowi (mamy nadzieję) uzyskamy sensowną selekcję. Stąd połączyliśmy funkcję *Permutation_importance* wraz z modelem *RandomForestClassifier* (którego hiperparametry wcześniej ustaliliśmy zgodnie z procedurą opisaną w następnym rozdziale). Wyniki tej procedury widoczne są na Rysunku 2. Prezentujemy je z pomocą boxplotów.

3.2 Wnioski o selekcji zmiennych

Wnioski z powyższej analizy są co najmniej zaskakujące. Okazało się, iż ponad 2/3 cech możemy uznać za nieistotne dla naszego zadania - uzyskania modelu o najlepszej zdolności predykcyjnej. Co więcej ich usunięcie znacząco poprawia jakość predykcyjną naszego modelu (wzrost *balanced_accuracy* o około 0.04). Stąd w dalszej części pracy będziemy rozważali uszczuploną ramkę danych. W szczególności zostawiamy jedynie oryginalne kolumny o numerach od 22 do 29 włącznie. Finalnie możemy przejść do tego co tygryski lubią najbardziej - dostrajania hiperparametrów :).

4 Eksperyment

W tym rozdziale opiszemy po krótku jak powstał Pipeline zaprezentowany w rozdziale **Konstrukcja**. Oczywiście nie obyło się bez licznych *GridSearchCV* - czyli przeszukiwań przestrzeni hiperparametrów poszczególnych modeli w celu znalezienia (przy pomocy pięciokrotnej krosvalidacji) ich optymalnych wartości dla danych obciętych zgodnie z procedurą z poprzedniego rozdziału. Z konieczności podziału pracy na etapy (która wynikała z tego, że o 9 rano otwierał się stok a autor jest zapalonym narciarzem) hiperparametry poszczególnych modeli były dostrajane oddzielnie. Wydawałoby się, że jednoczesna optymalizacja powinna skutkować lepszą dokładnością (w szczególności optymalizacja modelu zebranego już w *VotingClassifier*) jednak taki *GridSearch* 'liczyłby się' prawdopodobnie dłużej trwa okres obiegu Ziemi wokół Słońca... Ponadto dla każdego modelu *GridSearchCV* był przeprowadzany w kilku etapach. Najpierw siatka parametrów była ustawiana dość obszernie w celu wyłapania ogólnego trendu dla poszczególnych hiperparametrów. Następnie zawężaliśmy ją wokół otrzymanych w poprzedniej iteracji wartości i 'puszczaliśmy' jeszcze raz. Taka procedura była podyktowana staraniem minimalizacji czasu pojedynczego *GridSearchCV* oraz maksymalizacji jego skuteczności. Mamy świadomość, że jest to praktyka zbliżona do 'machania rękami'. Niemniej, okazała się dość skuteczna (a napewno w miarę szybka). Tak otrzymane dostrojone modele złączyliśmy w jeden komitet komitetów, czyli połączyliśmy je w *VotingClassifier* w nadziei na poprawę zdolności predykcyjnych.

5 Wyniki

Mamy już przygotowane do pracy dane, wytrenowany model, jedyne czego brakuje to jego ocena na zbiorze testowym. Jednak nie otrzymaliśmy etykiet dla zbioru testowego. Co za tym idzie oddajemy nasz model pod ocenę niejako 'w ciemno'. Naszym zadaniem było przygotowanie modelu o jak największej *balanced_accuracy*, czego de facto nie jesteśmy w stanie zmierzyć. Niemniej przez całość trwania przygotowania danych i eksperymentu staraliśmy się uzyskać model o jak największej 'zrównoważonej dokładności' (w szczególności oznaczało to zmianę parametru *scoring* właśnie na *balanced_accuracy* aby znaleźć model najlepiej wpasowujący się w tą metrykę). Ocena poszczególnych kroków podjętych procedur opierała się na danych treningowych, jednak mamy nadzieję, że dzięki krosvalidacji otrzymywanych rezultatów otrzymaliśmy coś 'sensownego'. Największą obawę stanowi ryzyko tego, iż nasz model został istotnie przeuczony (wiemy, że drzewa mają w naturze zbyt dokładne dopasowywanie się do danych treningowych oraz są znacząco niestabilne, mamy nadzieję jednak, iż użyte techniki temu zapobiegły). Wraz z tym raportem załączamy plik z wektorem prawdopodobieństw przynależności do klasy 1 dla danych testowych jakie zwrócił nasz model. Aby podać tu jednak faktycznie jakiegokolwiek wyniki prezentujemy jak nasz model wypadł podczas oceny za pomocą funkcji *cross_val_score*, która z pomocą pięciokrotnej krosvalidacji oceniła nasz model przy pomocy miary *balanced_accuracy* na zbiorze treningowym. Dla uciętych danych (zgodnie z wynikami selekcji

zmiennych) otrzymaliśmy dokładność równą ≈ 0.885 przy odchyleniu standardowym wynoszącym ≈ 0.02 (warto zaznaczyć, iż ten sam pomiar dla oryginalnych danych skutkował wynikami odpowiednio ≈ 0.841 i ≈ 0.03).

6 Podsumowanie

Pierwsze samodzielne, pełne i nieograniczone zadanie ML'owe było dużym wyzwaniem. Esencjonalna cecha uczenia maszynowego jaką jest niepewność co do otrzymanych rezultatów niejednokrotnie spędzała sen z powiek. Podjęte (trudne) decyzje co do wyboru modeli, ich parametrów czy istotnych dla zadania cech obserwacji, pomimo tego, że zostały podsunięte i 'zaakceptowane' przez program, często przyprawiały o ból głowy i stawiały pytania typu: czy da się lepiej? Odpowiedź najprawdopodobniej jest twierdząca, aczkolwiek mamy nadzieję, że zdobyta przez prawie semestr nauki wiedza pozwoliła chociaż zbliżyć się optymalnego rozwiązania. Niemniej ten projekt, jak i cały przedmiot *Wstęp do Uczenia Maszynowego*, napawiał optymizmem względem szeroko pojętego ML'a oraz jak najbardziej zachęcił do zgłębiania tego tematu w przyszłości. I choć niepewność jest istotną czy też porządaną cechą nowego adepta sztuki uczenia maszynowego, chcielibyśmy sobie życzyć trochę więcej stabilnego gruntu w nowych wyzwaniach, który zapewne uformuje się wraz z dalszym zgłębianiem tego ekscytującego tematu...

Dziękuję za uwagę!

7 Bibliografia

- 1 - https://scikit-learn.org/stable/auto_examples/preprocessing/plot_all_scaling.htmlplot-all-scaling-power-transformer-section
- 2 - https://scikit-learn.org/stable/auto_examples/ensemble/plot_forest_importances.html
- 3 - https://scikit-learn.org/stable/auto_examples/inspection/plot_permutation_importance.htmlsphx-glr-auto-examples-inspection-plot-permutation-importance-py